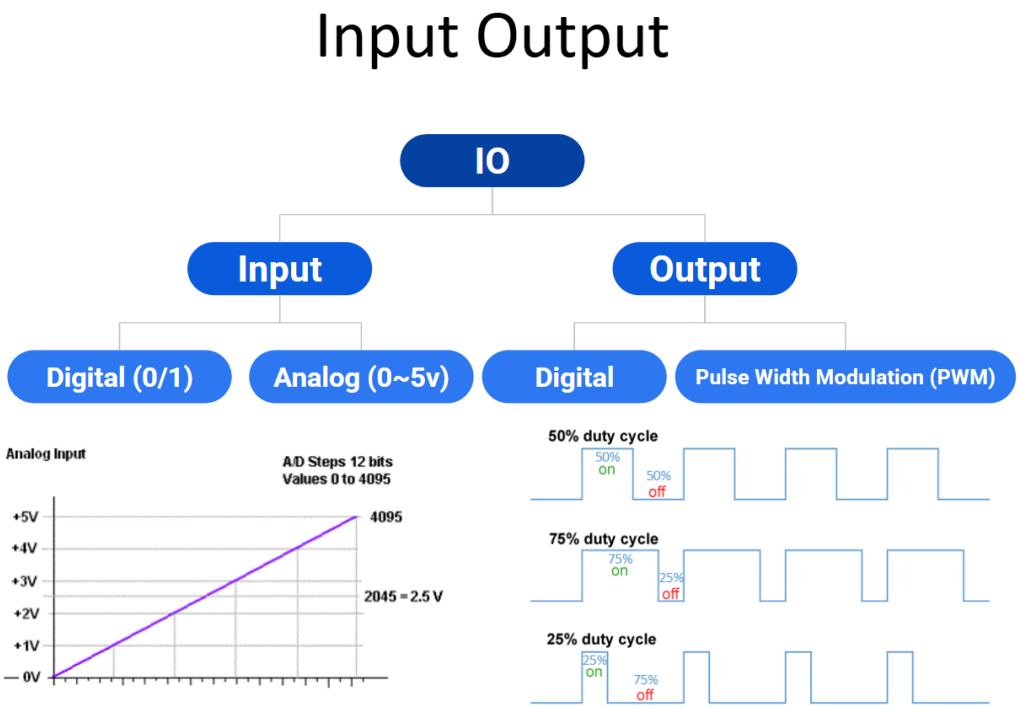
General Purpose IO (GPIO)
(Low level IO)



I/O Interfacing

I/O (Input/Output) devices, also known as peripherals, are essential components in any microprocessor-based or microcontroller-based system. These devices allow the system to communicate with the external world, enabling data exchange between the user and the processor.

Classification of I/O Devices

I/O devices are broadly classified into two main categories:

1. Input Devices

These devices are used to provide data or instructions to the system.

Examples:

- Keyboard
- Mouse
- Sensors
- Switches

They convert real-world signals or user actions into a form that the system can understand.

2. Output Devices

These devices are used to display or present the processed data from the system.

Examples:

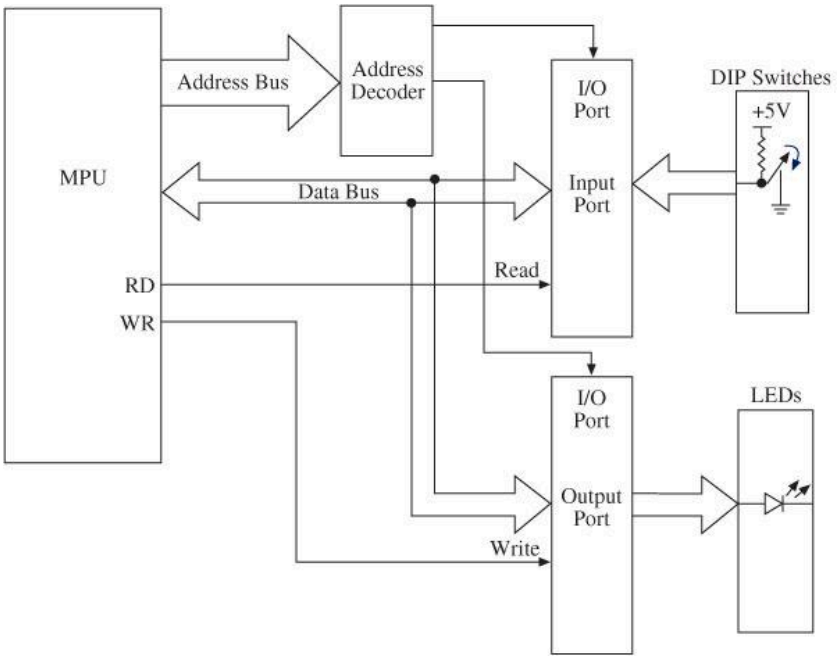
- LEDs
- LCD displays
- Buzzers
- Printers

They convert processed data into human-readable or observable form.

Block Diagram of I/O Interfacing

In a microcontroller or microprocessor system, communication with I/O devices happens through three main buses:

- Address Bus
- Control Bus
- Data Bus



Input Operation in I/O Interfacing

Step-by-Step Operation (Input)

When the system needs to take input from an input device, the following sequence occurs:

- The microcontroller places the I/O port address on the address bus.
- The address decoder selects the corresponding I/O port.
- The microcontroller activates the RD (Read) signal through the control bus.
- The input device sends data to the port.
- The port places the data on the data bus.
- The microcontroller reads the data from the data bus.

To read (receive) binary data from an input peripheral:

MPU places the address of an input port on the address bus, enables the input port by asserting the RD signal, and reads data using the data bus.

Data Flow

Input Device → I/O Port → Data Bus → Microcontroller

Output Operation in I/O Interfacing

When a microprocessor sends data to an output device (e.g., LED), the following steps occur:

Step-by-Step Operation (Output)

- The microprocessor places the I/O port address on the address bus.
- The address decoder selects the corresponding output port.
- The microprocessor places data on the data bus.
- The microprocessor activates the WR (Write) signal through the control bus.
- The selected port receives the data and transfers it to the output device.

To write (send) binary data to an output peripheral:

MPU places the address of an output port on the address bus, places data on data bus, and asserts the WR signal to enable the output port.

Data Flow

Microprocessor → Data Bus → I/O Port → Output Device

Buffer and Latch in I/O Interfacing

Buffer

- A buffer is used to temporarily hold data and drive signals without changing them.
- It helps in solving fan-out problems (not fan-in): meaning one output can safely drive multiple inputs.
- It also provides signal isolation and strength, but it does not perform noise cancellation or modify data.
- Maintains logic level (0 or 1).
- Improves signal driving capability.

Latch

- A latch is used to store (hold) a value (0 or 1) until it is changed or cleared.
- It is essential when we want the output to remain stable even if the input changes.
- Holds data until updated
- Used for stable output

Use in I/O Ports

Input Port → Buffer

- Input devices send data to the system through buffers.
- The buffer:
    - Transfers input data to the data bus
    - Protects the system from direct external signals

Output Port → Latch

- Output ports use latches.
- The latch:
    - Stores the data sent by the microprocessor
    - Keeps output stable (e.g., LED stays ON/OFF)

Mode 0 (Simple I/O Mode)

- Input lines are buffered
- Output lines are latched

Note

- Buffer = pass data safely (input side)

- Latch = hold data steadily (output side)

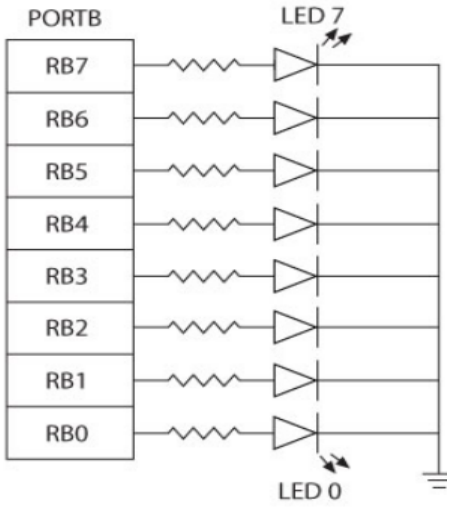
Output

- Light Emitting Diode (LED)
- Seven segment display (Alphanumeric)
- Matrix Display
- Liquid Crystal Display (LCD)
- Buzzer
- DC Motor
- Servo Motor
- Stepper Motor

Interfacing LED with I/O Ports

LEDs are commonly interfaced with microprocessors/microcontrollers to display output.
There are two standard methods of connecting LEDs:

Common Cathode Configuration (Current Sourcing)



Connection:

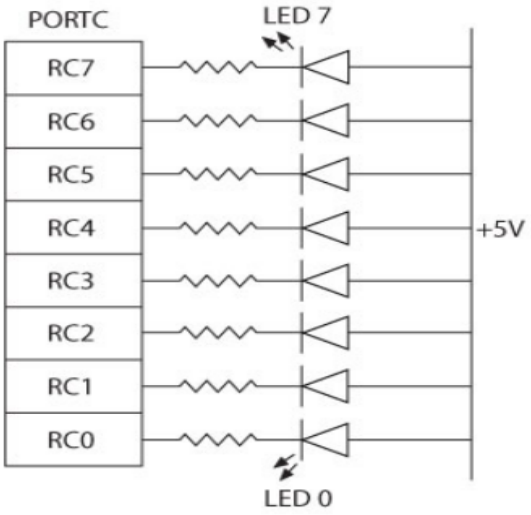
- All cathodes of LEDs are connected together and tied to Ground (0V).
- Each anode is connected to an I/O port pin through a current-limiting resistor.

Working:

- When the port outputs HIGH (1 / VCC):
  - Current flows from port → LED → ground
  - LED turns ON
- When the port outputs LOW (0):
  - No current flows
  - LED turns OFF

The microprocessor supplies current → called current sourcing.

Common Anode Configuration (Current Sinking)



Common Anode

Connection:

- All anodes of LEDs are connected together to VCC (+5V).
- Each cathode is connected to an I/O port pin through a resistor.

Working:

- When the port outputs LOW (0):
  - Current flows from VCC → LED → port (to ground)
  - LED turns ON
- When the port outputs HIGH (1):
  - No current flows
  - LED turns OFF

The microprocessor absorbs current → called current sinking.

Why are Resistors Needed?

- Limits current through LED
- Prevents damage to LED and microprocessor pins
- Typical value: 220Ω - 1kΩ

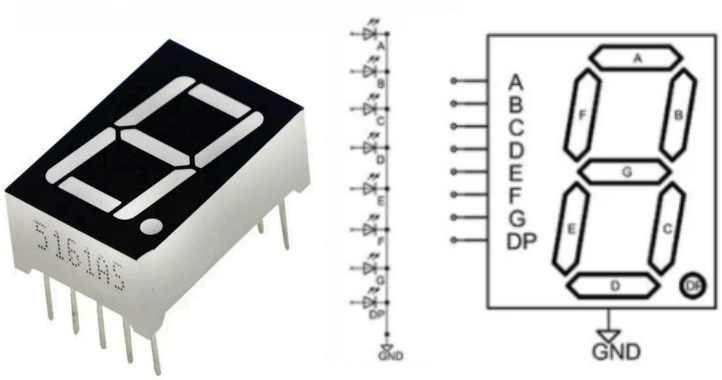
Comparison

Feature	**Common Cathode**	**Common Anode**
Common Terminal	Ground	VCC
LED ON Condition	Output = 1	Output = 0
Current Type	Sourcing	Sinking

Interfacing Seven-Segment Display

What is a Seven-Segment Display?

- A seven-segment display is made of 7 LEDs arranged in the shape of the digit “8”.
- An optional 8th segment (dp) is used as a decimal point.
- Each segment is labeled as:
  a, b, c, d, e, f, g (and dp for decimal point)

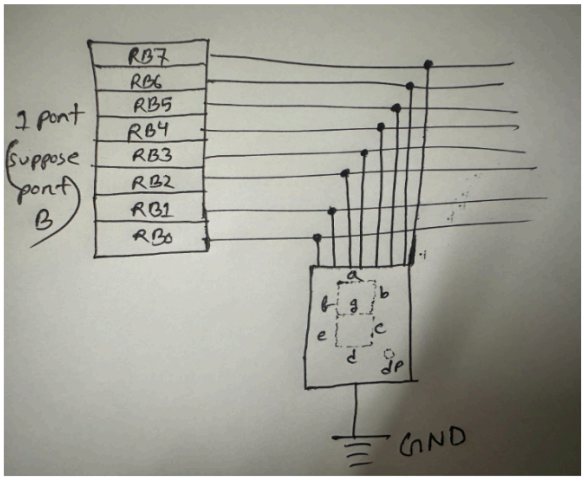
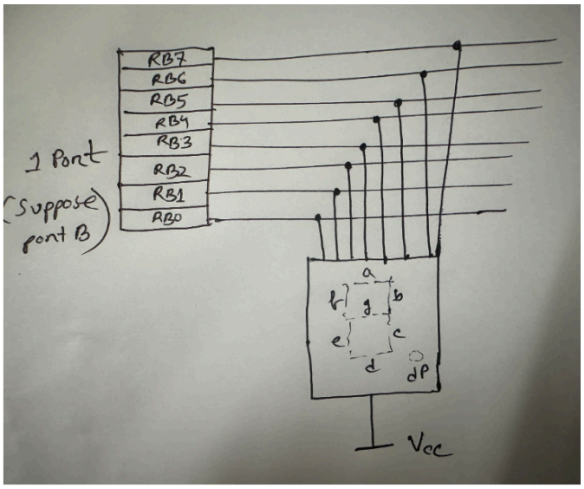


Purpose

- Used to display:
  - Decimal digits (0–9)
  - Some alphabets (limited, not all letters)

It mainly displays digits 0–9.

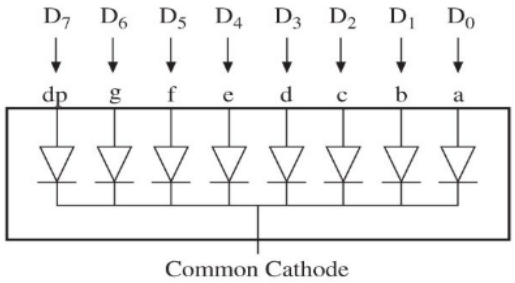
Types of Seven-Segment Displays

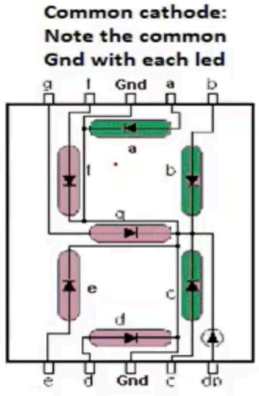
Common cathode mode	Common Anode mode
For 1 7-segment display 	For 1 7-segment display 
For multiple 7-segment display (Use transistor for controlling displays)	For multiple 7-segment display (Use transistor for controlling displays):

1. Common Cathode (CC)

- Segment ON → give 1 (HIGH)
- Segment OFF → give 0 (LOW)

Example: Display digit 3
Segments ON: a, b, c, d, g = 1

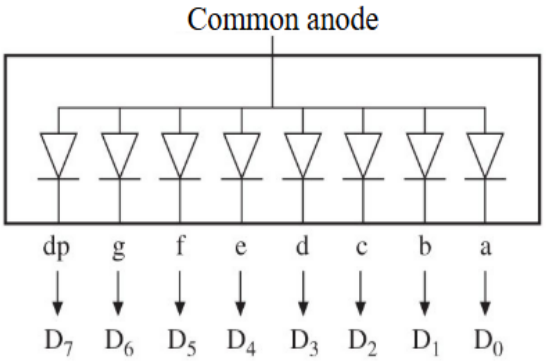


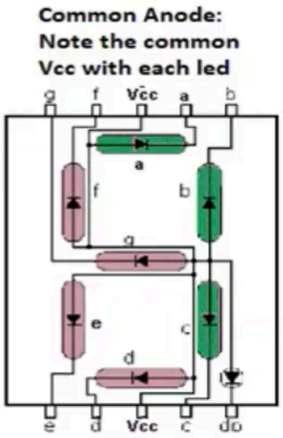


2. Common Anode (CA)

- Segment ON → give 0 (LOW)
- Segment OFF → give 1 (HIGH)

Example: Display digit 3
Segments ON: a, b, c, d, g = 0



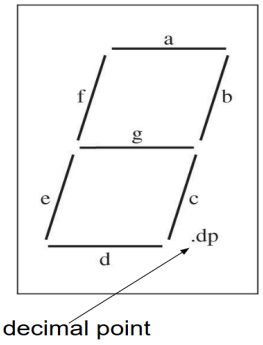


About Decimal Point (dp)

- When dp is included, it becomes 8 segments total
- Used for fractional numbers

Example

- To display 3.85, we need 3 separate displays



BCD to Seven-Segment Decoder

- Takes 4-bit Binary Coded Decimal (BCD) input
- Converts it into signals for 7 segments

Why 4-bit?

- Maximum decimal digit = 9
- Binary of 9 = 1001 (4 bits)

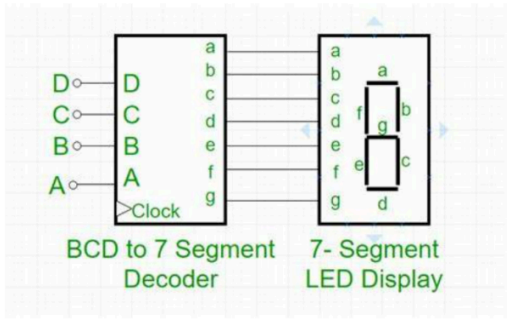
Working:

- Input: BCD (e.g., 0011 for 3)
- Output: Signals to segments → display 3

Note:

- Inputs 1010-1111 (10-15) are invalid in BCD
- Decoder usually:
    - Ignores them or
    - Produces blank/undefined output

BCD to 7-Segment Display table: [Common Cathode]



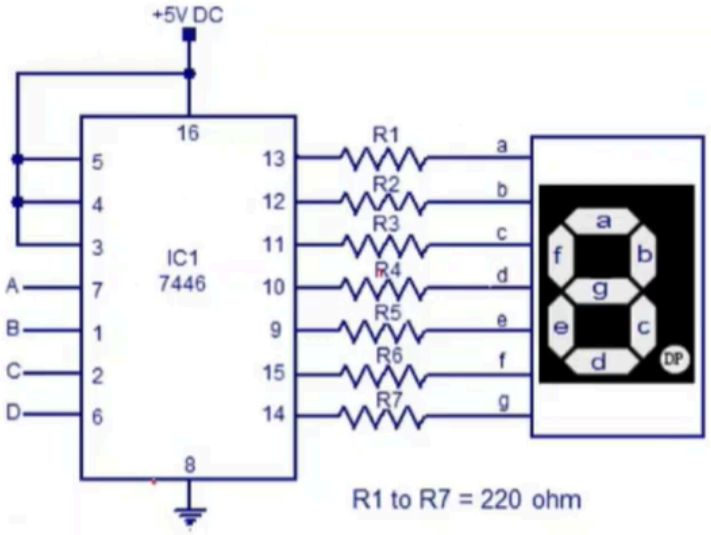
Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

Pins and Interfacing

- Total pins:
    - 7 segment pins (a-g) + 1 dp = 8 pins
        - Common pin(s)

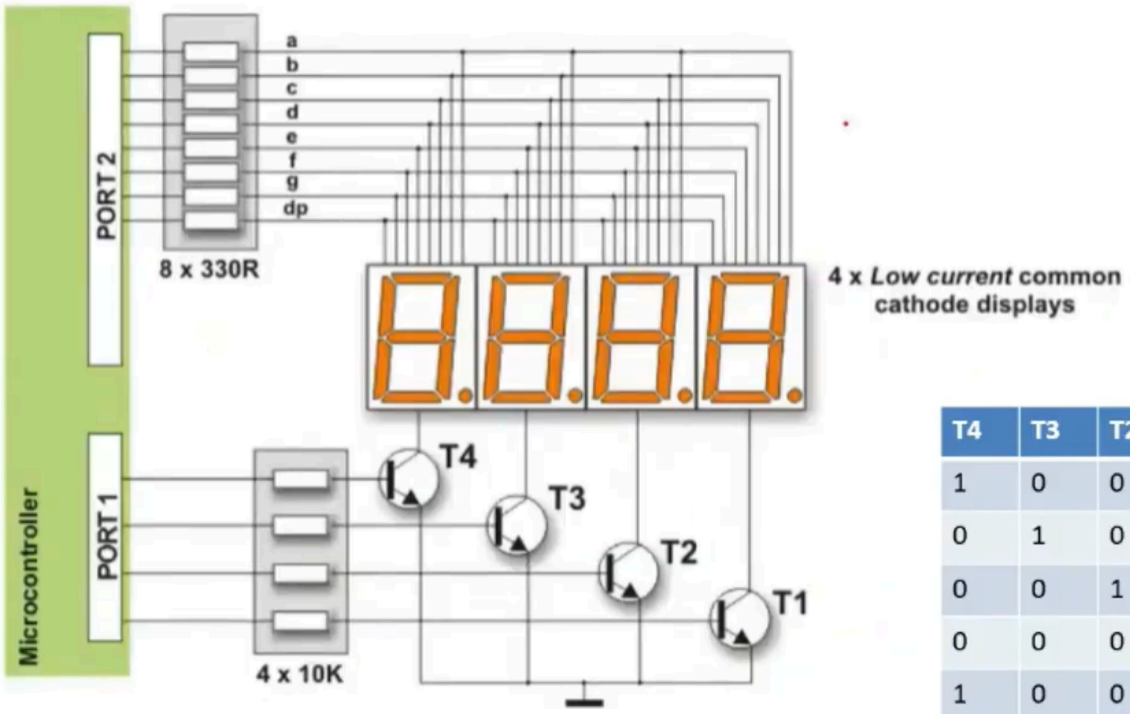
“use 10 pins” → Not always exact

- Typically 8-10 pins depending on design



Interfacing Multiple Seven-Segment Displays (Multiplexing)

This diagram shows how to control multiple 7-segment displays using fewer pins.



T4	T3	T2	T1
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

Why Are Two Ports Used?

Port 2 → Segment Control (a-g, dp)

- Connected to all segments (a-g + dp) of every display
- These lines are shared
- Send the segment pattern (not BCD directly!)

BCD → Decoder (or software) → segment signals (a-g)

Port 1 → Display Selection

- Used to select which display is ON
- Each pin controls one transistor (T1-T4)
- Only one display is active at a time

Role of Transistors

- Each transistor acts as a switch
- Controls connection between display and ground (for common cathode)

Working:

- If base = HIGH (1) → transistor ON
  → connects display to GND
  → display becomes active
- If base = LOW (0) → transistor OFF
  → display is disabled

Multiplexing Concept

- At any moment, only one display is ON
- Microcontroller:
  1. Sends segment data (Port 2)
  2. Activates one transistor (Port 1)
  3. Quickly switches to next display
- This happens very fast (milliseconds)

So human eyes see all displays ON simultaneously, “rotation works so fast that we see all 4” .

Without Multiplexing

- 1 display → 8 pins
- 4 displays → 8 × 4 = 32 pins

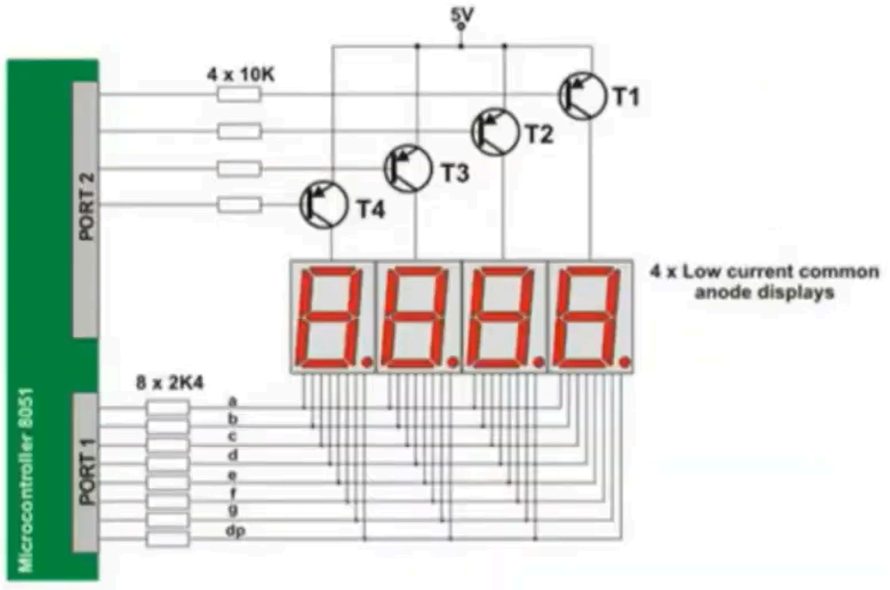
With Multiplexing (Huge reduction)

- Segment lines → 8 pins
- Display select → 4 pins
- Total = 12 pins

Note:

Port 2 = WHAT to display (segments)
Port 1 = WHERE to display (which digit)

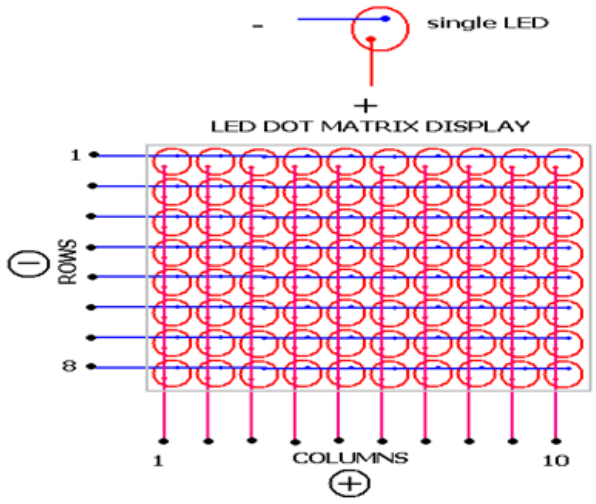
Common Anode Display

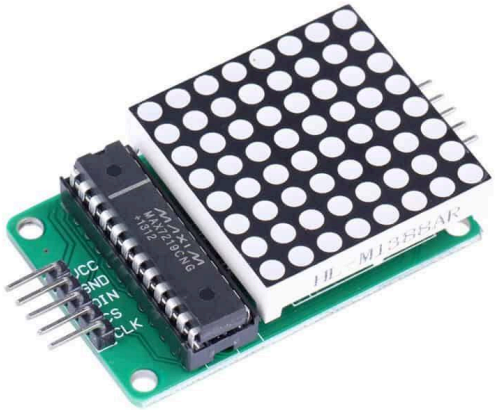


T4	T3	T2	T1
0	1	1	1
1	0	1	1
1	1	0	1
1	1	1	0
0	1	1	1

LED Matrix Display

An LED matrix display consists of 64 LEDs arranged in a 2D grid (8×8). It is used to display symbols, characters, and patterns.





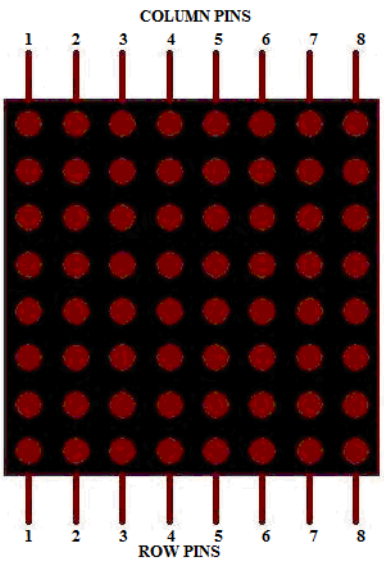
Structure

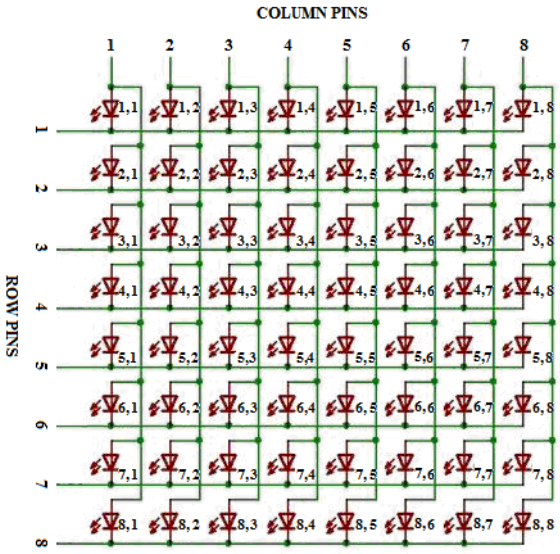
- The LEDs are arranged in rows and columns.
- Columns are connected to anodes.
- Rows are connected to cathodes.
- Each LED is identified by its row and column position.

Working Principle

To turn ON a specific LED at position (row, column):

- The corresponding row is made LOW (0 / GND).
- The corresponding column is made HIGH (1 / VCC).
- Current flows from column (anode) to row (cathode), and the LED turns ON.





Example:

- To turn ON LED at (4,6):
  - Row 4 → 0 (GND)
  - Column 6 → 1 (VCC)

Pin Requirement

- Total lines required:
  - 8 rows + 8 columns = 16 pins
- This results in high pin usage.

Reducing Pin Count Using a Johnson Counter

To reduce the number of pins, a Johnson Counter is used for row selection.

Port Configuration

- Port B (8 pins) → connected to columns (data lines)
- Port A (2 pins) → connected to Johnson Counter:
  - 1 pin → Enable
  - 1 pin → Clock

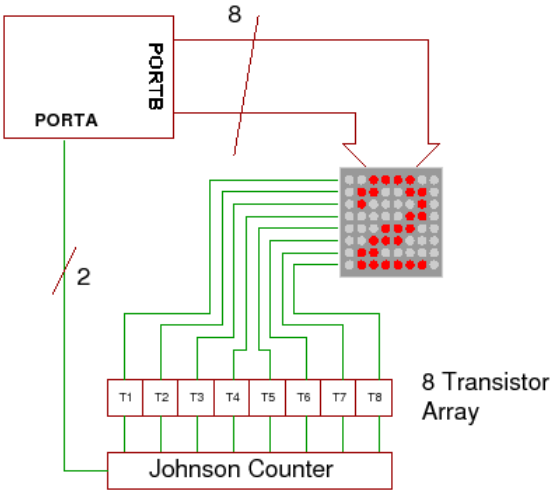
Total pins used = (8 + 2) = 10

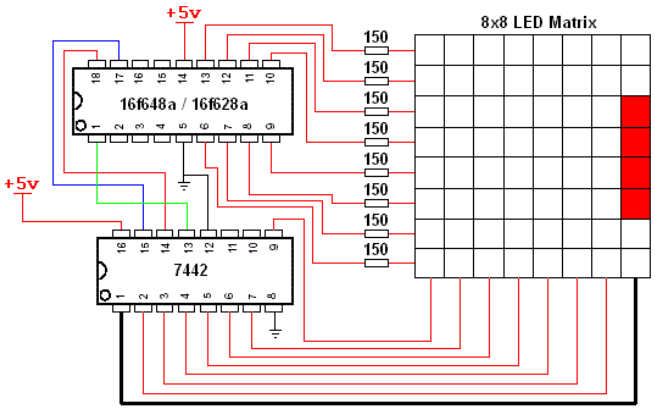
Johnson Counter Operation

- The Johnson Counter generates a sequence where one row is selected at a time.
- It is connected to an array of 8 transistors, each controlling one row.
- Only one transistor is active at a time, connecting that row to GND.

Row Activation

- When a row is selected:
    - Its cathode line is connected to GND
    - Other rows remain inactive





Multiplexing Process

1. Johnson Counter activates one row (sets it to GND)
2. Port B sends 8-bit data to columns
3. LEDs in that row turn ON based on column data
4. Clock pulse shifts activation to the next row
5. Process repeats for all rows

Example Data Flow

- If a row is active:
    - Sending 01100000 → specific LEDs in that row turn ON
    - Sending 00111000 → different LEDs turn ON

Each row receives its own column data when active.

Clock Pulse Requirement

- Total rows = 8
- Each clock pulse activates one row

Therefore, 8 clock pulses are required to refresh the full display once.

Note

- Rows → Selection (via Johnson Counter)
- Columns → Data (via Port B)
- Fast scanning → Complete visible image

LCD Interfacing (16×2 Character LCD with Built-in Controller)

A 16×2 LCD module can display 2 lines with 16 characters each, for a total of 32 characters. It is commonly interfaced with microcontrollers such as PIC18 using data and control lines.

Internal Organization

Display Data RAM (DDRAM)

- The LCD contains Display Data RAM (DDRAM).
- Each location stores 8-bit ASCII data.
- Each address corresponds to a specific position on the display.

Address Mapping

- Line 1 → addresses 0 to 15
- Line 2 → addresses 64 to 79 (not 16–31)

The controller maps these memory locations to physical positions on the LCD.

Working Principle

- The microcontroller sends an 8-bit ASCII code.
- This data is stored in DDRAM.
- The LCD controller converts ASCII into the corresponding character pattern.
- The character is displayed at the position corresponding to that DDRAM address.

Registers in LCD

The LCD has two internal registers:

1. Instruction Register (IR)

- Used to send commands (instructions)
- Controls LCD operation such as:
    - Clear display
    - Cursor position
    - Display ON/OFF
    - Entry mode

2. Data Register (DR)

- Used to send data (ASCII characters)
- Data written here is transferred to DDRAM and displayed

Control Signals

Three control pins are used:

- **RS (Register Select)**
  - RS = 0 → Instruction Register (command mode)
  - RS = 1 → Data Register (data mode)
- **RW (Read/Write)**
  - RW = 0 → Write operation
  - RW = 1 → Read operation
    (Usually kept LOW since microcontroller mostly writes)
- **E (Enable)**
  - Used to trigger data transfer
  - A HIGH-to-LOW pulse enables the LCD to read data

Power Connections

- Vcc → Power supply (+5V)
- GND → Ground
- VEE (or V0) → Contrast control (connected to variable resistor)

Data Pins

- D0-D7 (8 pins) used for data transfer
- Microcontroller sends data/commands through these pins

Data Writing Process

Step 1: Initialization (Command Mode)

- RS = 0
- RW = 0
- Send command via D0-D7

- Pulse Enable (E)
- LCD executes command

Commands configure:

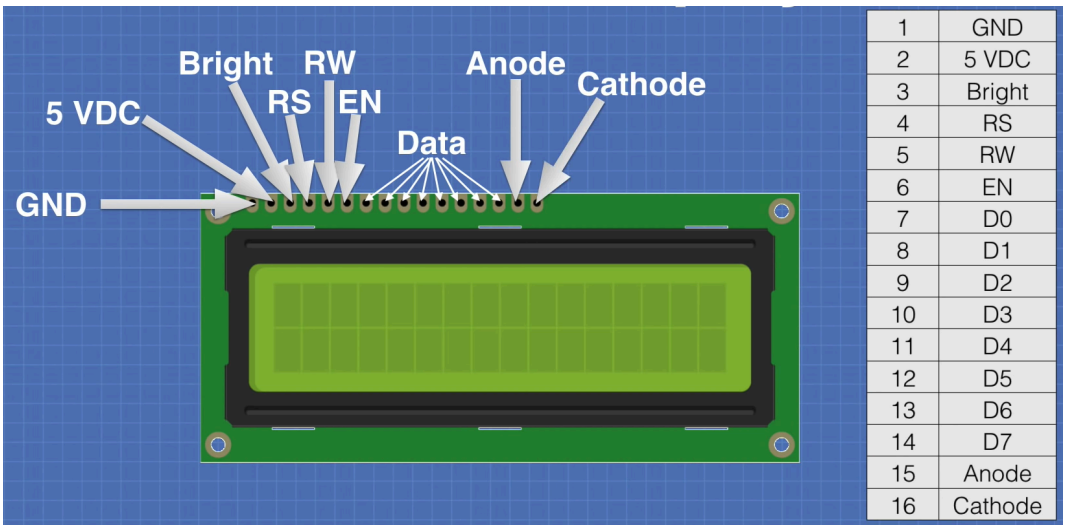
- Display format (16×2)
- Cursor position
- Entry mode

Step 2: Sending Data (Display Characters)

- RS = 1
- RW = 0
- Send ASCII data via D0-D7
- Pulse Enable (E)
- Character stored in LCD RAM and displayed

Data goes:
Microcontroller → Data Register → DDRAM → Display

LCD can be interfaced either in 8-bit or 4-bit mode



1	GND
2	5 VDC
3	Bright
4	RS
5	RW
6	EN
7	D0
8	D1
9	D2
10	D3
11	D4
12	D5
13	D6
14	D7
15	Anode
16	Cathode

8-bit Mode

- Uses all 8 data lines (D0-D7)

- Transfers 8 bits in one operation
- Faster
- Requires more pins

4-bit Mode

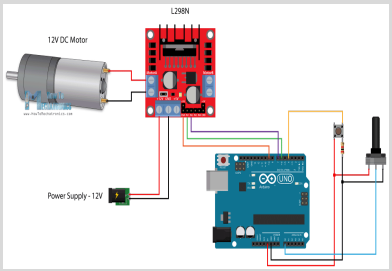
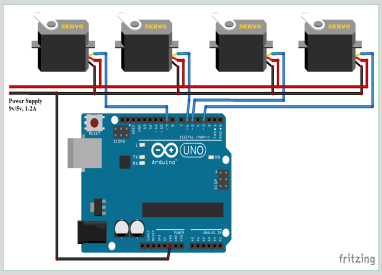
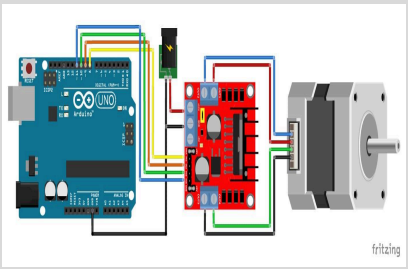
- Uses only 4 data lines (D4-D7)
- Transfers data in two steps (two Enable pulses):
    1. Higher 4 bits (MSB)
    2. Lower 4 bits (LSB)
- Slower
- Saves pins

Comparison

Feature	**8-bit Mode**	**4-bit Mode**
Data Lines	8	4
Speed	Faster	Slower
Pin Usage	More	Less
Transfer	1 step	2 steps

Note

- LCD displays characters using ASCII codes stored in DDRAM
- RS selects register, RW selects operation, E triggers transfer
- Commands configure LCD, data displays characters
- Two modes available: 8-bit (fast) and 4-bit (pin-saving)

DC Motor	Servo Motor	Stepper Motor
		
Continuous rotation	Limited / controlled rotation	Moves in steps
Speed controlled by voltage	Controlled by PWM signal	Controlled by pulses
No position control	Precise position control	Step-wise position control
No feedback	Uses feedback	No feedback
Simple structure	Complex structure	Medium complexity
Low accuracy	Very high accuracy	High accuracy
Low cost	High cost	Moderate cost
Used in fans, wheels	Used in robots, arms	Used in 3D printers, CNC

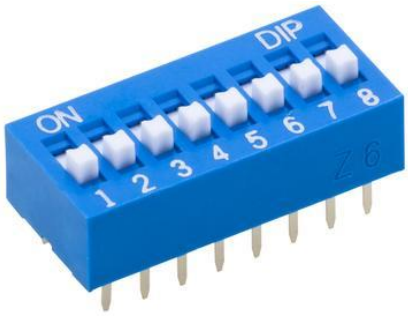
INPUT

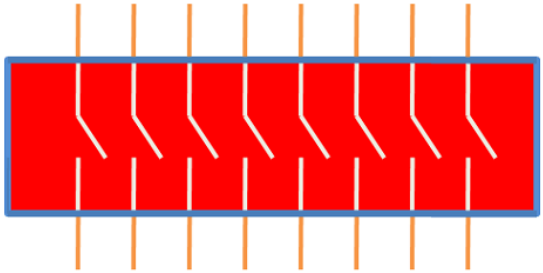
DIP Switch Interfacing

A DIP (Dual In-line Package) switch is used to provide digital input to a microcontroller or microprocessor system. Each switch represents one binary bit.

Circuit Configuration

- The circuit contains 8 switches connected to Port B pins (RB0-RB7).
- Each input line is connected to:
    - +5V through a 10kΩ pull-up resistor
    - Ground through the switch





DIP Switch Schematic

Pull-Up Resistor

The pull-up resistor keeps the input at logic HIGH (1) when the switch is open.

- Switch open → input connected to +5V through resistor → logic 1
- Switch closed (pressed) → input connected to GND → logic 0

The resistor prevents direct short circuit between VCC and GND when the switch is pressed.

Working Principle

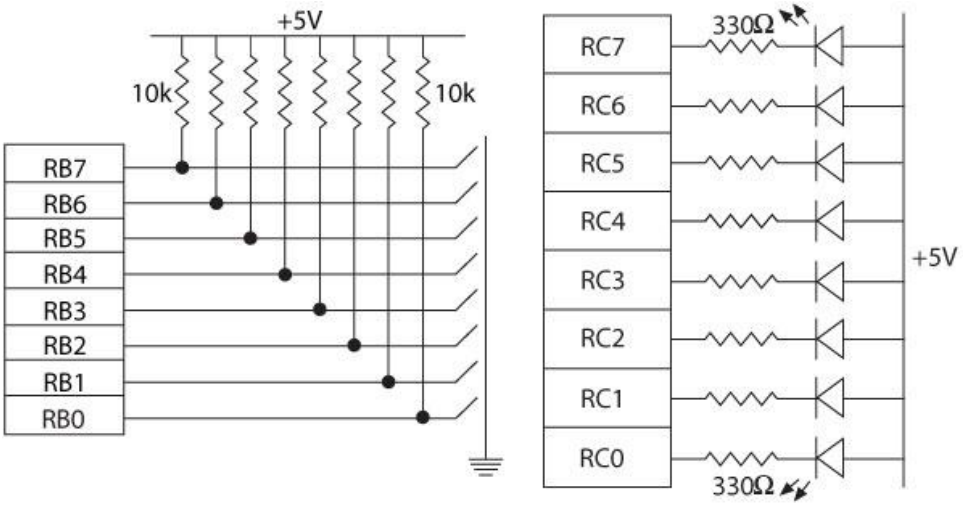
Switch Not Pressed

- Circuit path to GND is open
- Input pin receives +5V through pull-up resistor
- Port pin reads logic **1**

Switch Pressed

- Input pin directly connected to GND
- Port pin reads logic **0**

Thus, the switch operates in active LOW configuration.



Example

If switches connected to:

- RB0 and RB3 are pressed

Then:

- RB0 = 0
- RB3 = 0
- Remaining pins = 1

Binary pattern:
11110110
(depending on bit order)

Key Concept

- Open switch → HIGH (1)
- Pressed switch → LOW (0)

This method is commonly used because it provides stable input logic levels and avoids floating inputs.

Push-Button Switch Interfacing

A push-button switch is a temporary input device used to provide digital input to a microcontroller or microprocessor system.

The connection is similar to a DIP switch, but the contact is momentary. The switch remains active only while it is being pressed.

Circuit Configuration

- One side of the switch is connected to:
    - +5V through a pull-up resistor
    - Input port pin
- The other side is connected to:
    - Ground (GND)

Working Principle

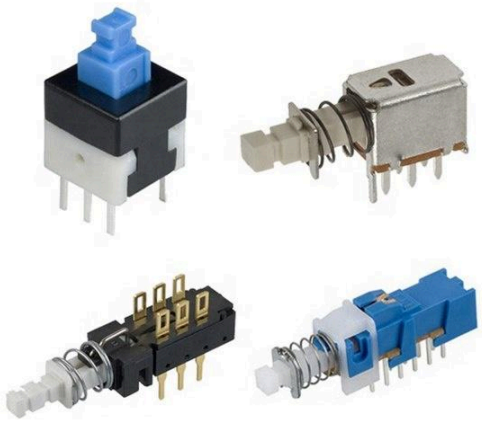
Button Not Pressed

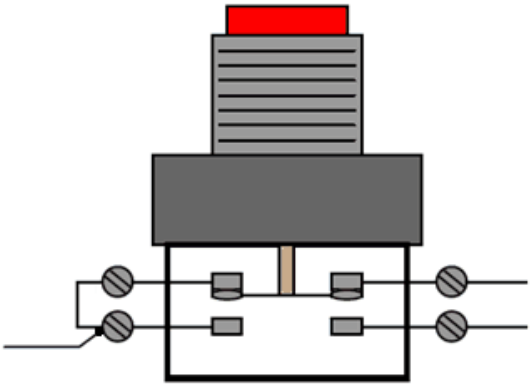
- Input pin receives +5V through the pull-up resistor
- Port pin reads logic **1**

Button Pressed

- Input pin becomes connected to GND
- Port pin reads logic 0

Thus, the push-button switch usually works in active LOW configuration.





Key Debouncing

Push-button switches have a problem called key debouncing.

When a key is pressed or released, the mechanical metal contacts do not settle immediately. During this short time, the signal rapidly changes between logic 0 and logic 1, producing a distorted signal called bounce.

As a result, a single key press may be detected as multiple inputs by the microcontroller.

Bouncing Signal

If the key is not fully settled after pressing, the output may appear as:

$0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0$

instead of a single stable transition.

This unwanted fluctuation is called the key debounce problem.

Debouncing Techniques

The multiple readings caused by bouncing can be eliminated using:

- Software debouncing
- Hardware debouncing

1. Software Key Debouncing

In software debouncing, the microcontroller waits for a short time after detecting a key press.

Steps

1. Detect key press
2. Wait for approximately 20 ms
3. Read the port again
4. If the input is still active, confirm the key press

Example:

- If the port value is still less than FFH, the key is considered pressed.

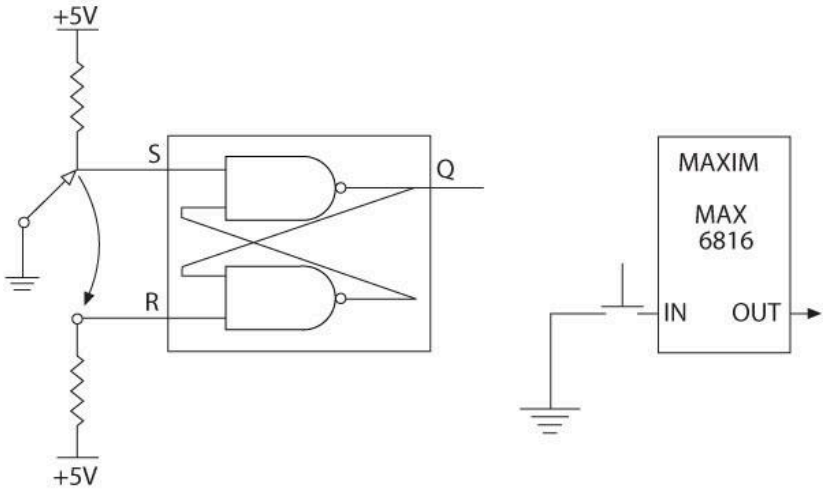
The delay allows the bouncing signal to settle before processing the input.

2. Hardware Key Debouncing

Hardware debouncing uses electronic circuits to remove bouncing and produce a stable signal.

It is based on:

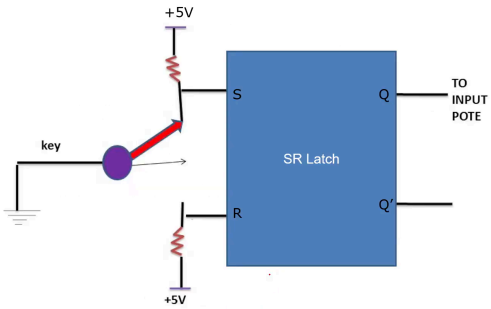
- Generating a delay
- Switching logic levels at a threshold value



S-R Latch Debouncing

An S-R latch can eliminate bouncing and generate a clean output pulse.

Working Principle



- When the key is pressed:
  - S = 0
  - R = 1
  - Output changes state
- The latch holds this output even if bouncing occurs.
- When the key is released:
  - R = 0
  - Output returns to logic 1

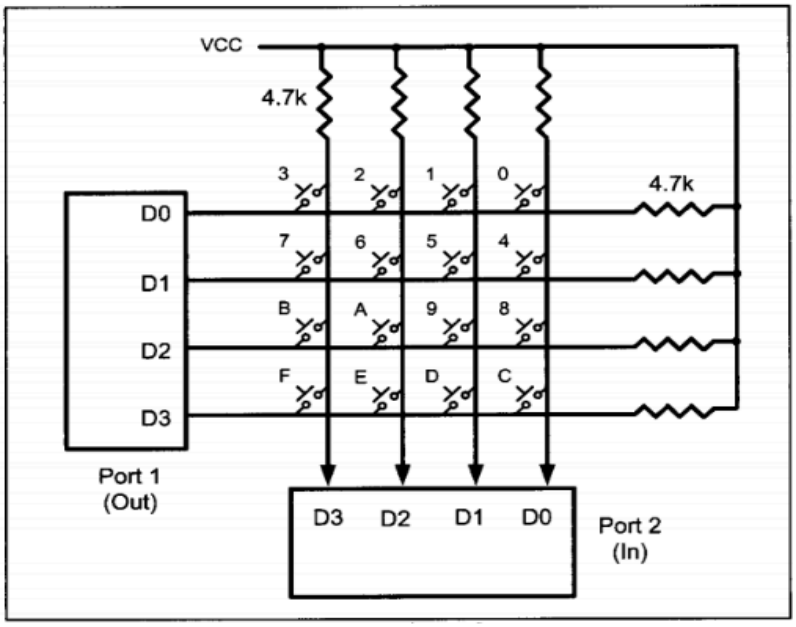
Thus, the S-R latch prevents multiple transitions caused by contact bouncing.

Interfacing a Matrix Keyboard

A matrix keyboard is arranged in rows and columns to reduce the number of I/O pins required.

The keyboard is scanned by grounding one row (or column) at a time and checking the corresponding columns (or rows).

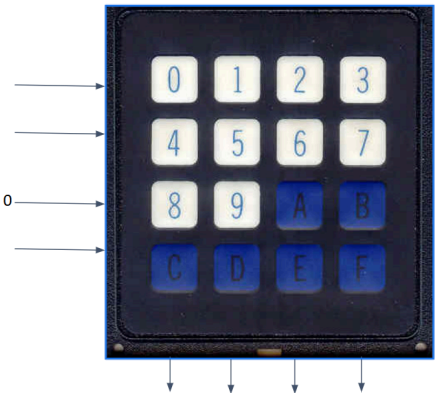
Once a key is detected, the key is identified using its row-column intersection.



Keyboard Layout

Example 4×4 matrix keyboard:

Row	Keys
R0	0 1 2 3
R1	4 5 6 7
R2	8 9 A B
R3	C D E F



Assembly table:

R0 DB 00H,01H,02H,03H
R1 DB 04H,05H,06H,07H

R2 DB 08H,09H,0AH,0BH
R3 DB 0CH,0DH,0EH,0FH

KEYPRESSED DB 0

82C55 Configuration

Before scanning the keyboard, the 82C55 Programmable Peripheral Interface must be configured.

MOV AL, 82H
OUT CWR, AL

Meaning of Control Word 82H

- Port A → Output
- Port B → Input
- Mode 0 operation

Typically:

- Rows connected to Port A
- Columns connected to Port B

Control Word Register (CWR)							
8				2			
1	0	0	0	0	0	1	0
I/0	MODE 0 PORT A		PORT A Output		MODE 0 PORT B	PORT B Input	

Working Principle

The keyboard scanning process has three steps:

1. Column identification
2. Row identification
3. Key identification

1. Column Identification

Initially:

- All column lines are connected to VCC
- Therefore:
  D3 D2 D1 D0 = 1111

When a key is pressed:

- One column becomes connected to a LOW row line
- The corresponding column bit changes to 0

Example:

- If key “9” is pressed:
  - Column pattern becomes:
    1101

This indicates:

- Column 2 is active

The detected column value is stored for later use.

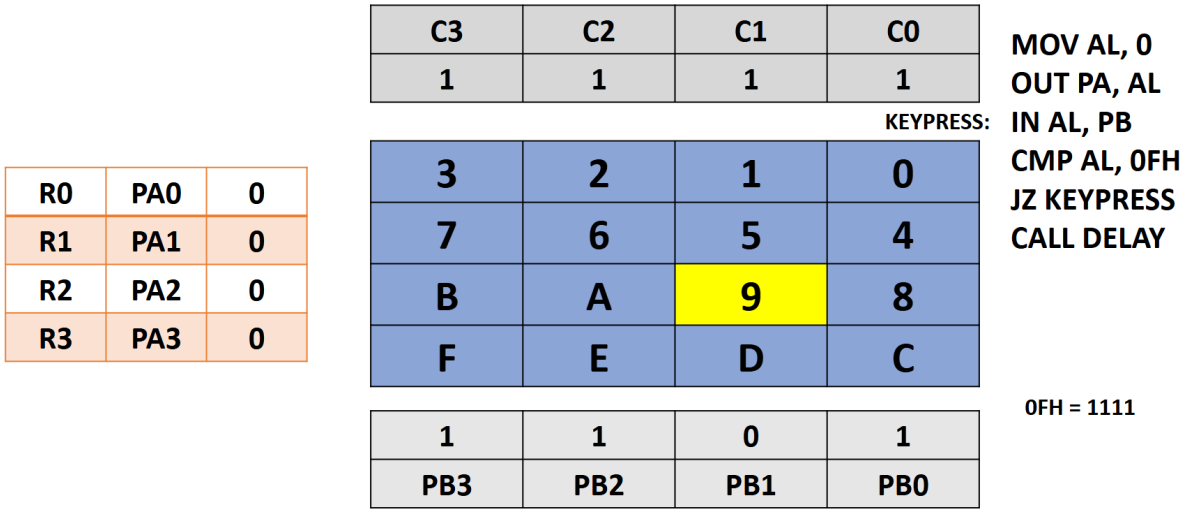
Key Detection Program

```
KEYPRESS:
   IN AL, PB
   CMP AL, 0FH
   JZ KEYPRESS
   CALL DELAY
```

Explanation

- Read Port B
- Compare with 0FH (1111)
- If all columns are HIGH:
    - no key pressed
- Otherwise:
    - key press detected

Delay is used for debouncing.



2. Row Identification

Rows are activated one at a time.

The microprocessor sends:

- one LOW (0)
- remaining HIGH (1)

to identify which row contains the pressed key.

Row Scanning Example

Scan Row 0

```
MOV AL, 1110B
OUT PA, AL

IN AL, PB
CMP AL, 0FH
JZ R1

LEA SI, R0
JMP COL
```

Explanation

- 1110:
  - R0 = 0
  - R1,R2,R3 = 1

If a column changes from 1 to 0:

- key exists in Row 0

Otherwise:

- continue scanning next rows

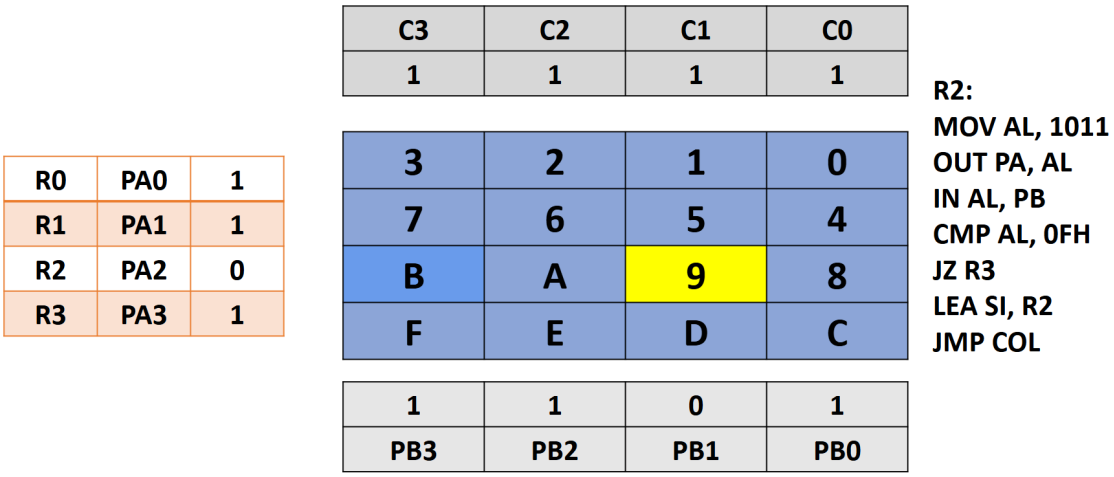
Example of Row Detection

Suppose:

- Column pattern = 1101
- Active row = R2

Then:

- Key is located at intersection:
  - Row 2
  - Column 1



3. Key Identification

After row and column are identified:

- The program accesses the corresponding row table
- Finds which column contains 0
- Determines the key value

Key Lookup

COL:
MOV AL, [SI]

The program may use:

- SHR (Shift Right)
- masking operations

to detect the position of 0 in the column pattern.

Example

Detected:

- Row = R2

- Column pattern = 1101

Column 2 is active.

From table:

Row2
B A 9 8

Column 2 corresponds to:

- Key = 9